CHAPTER FOUR

SYSTEM IMPLEMENTATION

4.0 Overview

This chapter presents the implementation of the Community Incident Reporting System (CIRS). It describes the software development tools and platforms used in building the system, the database design and normalization process, the Entity Relationship Diagram (ERD), and the user interface design. The implementation translates the system requirements and design models presented in the previous chapter into a fully functional web-based application capable of supporting community incident reporting and management.

4.1 Development Tools and Platform

The Community Incident Reporting System was implemented using modern web development technologies selected based on their scalability, reliability, ease of maintenance, and compatibility with geospatial technologies.

The major development tools are discussed below.

4.1.1 React.js

React.js was used as the primary frontend technology for developing the user interface of the application. React is an open-source JavaScript library developed by Meta for building dynamic, interactive, and responsive web applications. It adopts a component-based architecture, where the user interface is divided into independent, reusable components that can be developed, tested, and maintained separately. This modular approach simplifies application development and improves code organization.

React was selected for this project for several reasons. First, it supports component-based development, allowing different parts of the application, such as navigation bars, forms, tables,

dashboards, and modal windows, to be built as separate components. These components can be reused across multiple pages, reducing code duplication and ensuring consistency throughout the system.

Secondly, React enables the creation of reusable user interface components, which significantly improves development efficiency. Once a component has been created and tested, it can be incorporated into different sections of the application without rewriting the same code. This not only reduces development time but also simplifies future enhancements and maintenance.

Another important reason for selecting React is its Virtual Document Object Model (Virtual DOM). Unlike traditional web applications that update the entire webpage whenever a change occurs, React updates only the portions of the interface that have changed. This selective rendering improves application performance, reduces unnecessary processing, and provides users with a faster and smoother browsing experience, especially when handling large datasets or frequent user interactions.

React also improves the maintainability and scalability of the application. Its well-structured architecture allows developers to easily identify, modify, or extend specific components without affecting other parts of the system. As new features or modules are added in the future, they can be integrated with minimal changes to the existing codebase.

Furthermore, React integrates seamlessly with RESTful APIs, making it suitable for communicating with the backend server. Through asynchronous HTTP requests, the frontend can efficiently send user requests, retrieve data from the server, and display updated information without requiring full page reloads. This capability enhances the responsiveness and overall usability of the application.

Finally, React facilitates the development of responsive web applications that adapt to different screen sizes and devices. Combined with responsive CSS frameworks and modern web design

techniques, the application provides a consistent user experience across desktop computers, laptops, tablets, and smartphones. This ensures that users can conveniently access the system regardless of the device they are using.

Overall, React.js was an appropriate choice for the frontend development of this project because it offers high performance, code reusability, scalability, ease of maintenance, and excellent integration with backend services, all of which contribute to the development of a modern and efficient web-based application.

4.1.2 Laravel Framework

Laravel was used as the backend framework for developing the server-side components of the application. Laravel is a powerful open-source PHP framework designed to simplify web application development by providing a robust set of tools and libraries for handling common backend tasks. It follows modern software development principles and encourages the development of secure, scalable, and maintainable applications.

One of the primary reasons for selecting Laravel is its implementation of the Model-View-Controller (MVC) architectural pattern. The MVC architecture separates the application's business logic, user interface, and data management into distinct components. This separation improves code organization, enhances readability, and makes the application easier to develop, test, and maintain. It also allows multiple developers to work on different parts of the system simultaneously with minimal conflict.

Laravel also provides secure authentication and authorization mechanisms for managing user access to the system. Built-in authentication features simplify the implementation of user registration, login, password management, and session handling while protecting sensitive user information. Authorization features ensure that users can only access resources and perform

actions that correspond to their assigned roles and permissions, thereby improving the overall security of the application.

Another significant advantage of Laravel is its routing system, which provides a clean and organized way of defining application endpoints. Routes map incoming HTTP requests to specific controller methods, making it easier to manage application functionality and develop RESTful APIs. The routing mechanism also supports middleware, enabling additional features such as authentication, authorization, and request filtering before requests reach the application's business logic.

Laravel includes powerful request validation capabilities that ensure user input is properly verified before being processed or stored in the database. Validation rules help prevent invalid or malicious data from entering the system, thereby improving data quality, maintaining database integrity, and enhancing application security.

The framework also provides excellent support for RESTful API development. Laravel simplifies the creation of API endpoints that enable seamless communication between the frontend developed in React.js and the backend server. Through these APIs, data can be exchanged efficiently using JSON, allowing the frontend to perform operations such as creating, retrieving, updating, and deleting records without requiring page reloads.

Another important feature is database migration support, which allows developers to define and manage database structures using PHP code instead of manually writing SQL scripts. Migrations provide version control for the database schema, making it easier to create, modify, and maintain tables consistently across different development and deployment environments. This ensures that all developers working on the project use the same database structure.

Laravel further enhances development through its integrated features such as the Eloquent Object-Relational Mapping (ORM), middleware, caching, logging, exception handling, and task

scheduling. These built-in tools reduce the amount of repetitive code developers need to write while improving application performance, reliability, and maintainability.

Overall, Laravel was selected because it provides a comprehensive and structured framework for backend development. Its support for the MVC architecture, secure authentication, routing, request validation, RESTful API development, database migrations, and numerous built-in development tools enabled the development of a secure, efficient, and maintainable backend capable of supporting the functional requirements of the system.

4.1.3 MySQL Database

MySQL was used as the relational database management system (RDBMS) for storing and managing all application data. It is one of the most widely used open-source database systems due to its reliability, performance, security, and ease of integration with modern web development frameworks. MySQL provides an efficient environment for storing structured data while supporting fast data retrieval and maintaining data integrity through relational database principles.

The database serves as the central repository for all information generated and managed by the application. It stores user accounts, including user profiles, authentication credentials, and role information required for secure access to the system. This enables the application to authenticate users and enforce role-based access control.

The database also stores incident reports submitted by users. Each report contains relevant details such as the incident title, description, date and time of occurrence, location information, category, severity level, current status, and the identity of the reporting user. Maintaining this information in a structured database allows efficient retrieval, updating, and analysis of incident records.

To ensure proper organization of reported incidents, MySQL stores incident categories, which classify incidents into predefined groups. This categorization simplifies data management, enables faster searching and filtering, and supports statistical analysis and reporting.

The system further stores information about uploaded images associated with incident reports. Rather than storing image files directly within the database, MySQL maintains references such as filenames, storage paths, and related metadata. These records establish relationships between images and their corresponding incident reports, enabling users and administrators to access supporting visual evidence whenever required.

The database also maintains the status of incidents, allowing administrators to monitor the progress of each reported case. Status values such as *Pending*, *Under Review*, *In Progress*, and *Resolved* enable effective tracking of incident resolution and provide users with updates on the handling of their reports.

In addition, MySQL stores notifications generated by the system. These notifications inform users and administrators about important events, including new incident submissions, status updates, assigned tasks, and other system activities. Storing notification records ensures that users can review previous alerts whenever necessary.

Since the application incorporates location-based reporting, MySQL stores geographical coordinates, including latitude and longitude values for each reported incident. These coordinates facilitate accurate mapping of incidents, spatial analysis, and visualization using mapping services, enabling authorities to identify incident hotspots and respond more effectively.

MySQL was selected for this project for several reasons. It is reliable, offering stable performance and robust transaction management for applications that require continuous access to data. As an open-source database management system, it is freely available and supported by a large global community, making it a cost-effective solution for academic and commercial projects alike.

The database is also recognized for its high performance, providing fast execution of queries and efficient management of large volumes of structured data. This ensures that users experience quick response times when retrieving or updating information within the application.

Another important advantage of MySQL is its seamless integration with the Laravel framework. Laravel provides native support for MySQL through its Eloquent Object-Relational Mapping (ORM) and database migration features, simplifying database operations, reducing development effort, and improving code readability.

Finally, MySQL is highly effective in managing relational data through the use of tables, primary keys, foreign keys, indexes, and constraints. These relational features help enforce data integrity, eliminate redundancy, and maintain consistency among related entities within the system. Such capabilities are essential for applications involving interconnected data such as users, incidents, categories, notifications, and geographical information.

Overall, MySQL was selected because it provides a secure, reliable, high-performance, and scalable database solution that integrates seamlessly with Laravel while efficiently managing the relational data required by the application.

4.1.4 Google Maps API

Google Maps API was integrated into the application to provide geospatial and location-based functionality. It serves as a key component for visualizing and interacting with geographic data related to reported incidents. By leveraging Google's mapping services, the application is able to present accurate and interactive map-based representations of incident locations.

The major functions provided by the Google Maps API within the system include the display of incident locations, where each reported incident is plotted on an interactive map for easy

visualization. This allows users and administrators to quickly identify where incidents have occurred within a given area.

The API also supports map markers, which are used to highlight specific incident points. Each marker represents an individual incident and can be interacted with to view additional details such as description, status, and time of reporting.

Another important feature is reverse geocoding, which converts geographic coordinates (latitude and longitude) into readable location names. This functionality enhances user experience by allowing the system to translate raw coordinate data into meaningful addresses or place names.

In addition, the Google Maps API enables map navigation, allowing users to zoom, pan, and explore different regions seamlessly. This improves usability and ensures that users can effectively interact with spatial data.

The system also utilizes hotspot visualization, which helps identify areas with a high concentration of incidents. This feature is particularly useful for administrators, as it supports decision-making by highlighting regions that may require urgent attention or resource allocation.

Overall, the Google Maps API provides accurate, real-time mapping services that enhance situational awareness and improve the ability of administrators to analyze and respond to incidents effectively.

4.1.5 Visual Studio Code

Visual Studio Code (VS Code) was used as the primary Integrated Development Environment (IDE) for the development of the application. It is a lightweight yet powerful source code editor developed by Microsoft, widely used for modern web and software development.

VS Code was selected for this project due to its lightweight performance, which allows it to run efficiently even on systems with limited resources while still providing advanced development features. Its fast startup time and smooth operation make it suitable for continuous development activities.

Another reason for its selection is its strong support for both Laravel and React.js development. Through extensions, VS Code provides syntax highlighting, code formatting, IntelliSense, and project scaffolding tools that significantly improve development efficiency and reduce coding errors.

The editor also integrates seamlessly with Git version control, enabling developers to commit changes, track modifications, and manage branches directly from the IDE. This improves workflow efficiency and supports collaborative development.

VS Code is also known for its intelligent code completion (IntelliSense) feature, which provides real-time suggestions, parameter hints, and documentation while coding. This helps developers write cleaner and more accurate code in less time.

In addition, VS Code supports a wide range of debugging extensions, allowing developers to identify and fix errors efficiently within both frontend and backend codebases. This improves system reliability and reduces development time.

Overall, Visual Studio Code was selected because it provides a flexible, efficient, and feature-rich development environment suitable for building full-stack applications using React.js and Laravel.

4.1.6 Git and GitHub

Git was used as the version control system for managing the source code of the application, while GitHub was used as the remote repository hosting platform. Together, these tools provided a

structured and reliable way to manage changes in the codebase throughout the development lifecycle.

One of the key benefits of using Git is source code management, which allows developers to track changes made to files over time. This makes it possible to review previous versions of the code, identify modifications, and maintain a clear development history.

Git and GitHub also supported collaborative development, enabling multiple developers to work on the project simultaneously without overwriting each other's changes. Through branching and merging, different features could be developed independently and later integrated into the main project.

Another important feature is the ability to rollback to previous versions of the application. In cases where errors or issues are introduced, Git allows the system to revert to a stable version, ensuring continuity and reducing downtime during development.

Additionally, GitHub served as a secure platform for project backup and remote storage, ensuring that the source code is safely stored online and can be accessed from any location. This reduces the risk of data loss due to system failure or accidental deletion.

Overall, Git and GitHub played a critical role in ensuring efficient version control, collaboration, and safe management of the application's source code.

4.1.7 Web Browser

Google Chrome was used as the primary web browser during the development and testing of the application. It provided a stable and reliable environment for running and evaluating both the frontend and backend components of the system.

Chrome Developer Tools played an essential role in the development process by providing a range of debugging and testing capabilities. These tools were used for debugging, allowing developers to inspect HTML elements, monitor JavaScript execution, and identify errors in real time.

The browser was also used for API testing, enabling verification of responses from backend endpoints and ensuring correct communication between the frontend and backend systems.

In addition, Chrome Developer Tools supported performance monitoring, which helped evaluate page load times, resource usage, and overall application efficiency. This ensured that the system maintained acceptable performance standards.

Finally, the browser was used for responsive design testing, allowing the application to be tested across different screen sizes and device simulations. This ensured that the user interface remained consistent and usable on desktops, tablets, and mobile devices.

Overall, Google Chrome provided a comprehensive environment for testing, debugging, and validating the functionality and responsiveness of the application throughout the development process.

Table 4.1 Development Tools

Tool	Purpose
React.js	Frontend development
Laravel	Backend API and business logic
MySQL	Database management
Google Maps API	GIS and mapping
Visual Studio Code	Code development
Git/GitHub	Version control
Google Chrome	Testing and debugging

4.2 Database Design

The Community Incident Reporting System is built on a relational database model designed in accordance with standard database normalization principles. The main objective of the database design is to ensure efficient data storage, eliminate redundancy, and maintain consistency across all system operations. By applying normalization techniques, the database structure was organized in a way that reduces duplication of data while improving data integrity and scalability.

The database design ensures several key qualities that are essential for a reliable information system. These include minimal redundancy, where data is stored only once and referenced where necessary to avoid duplication. This helps reduce storage overhead and prevents inconsistencies that may arise from repeated data entries.

It also ensures data consistency, meaning that all related data across different tables remains synchronized. Any updates made to a record are automatically reflected in related tables through relational constraints, ensuring that the system always presents accurate and up-to-date information.

Another important aspect is data integrity, which is enforced through the use of primary keys, foreign keys, and database constraints. These mechanisms ensure that only valid and properly linked data is stored in the system, preventing orphan records and invalid relationships between entities.

The design further supports efficient querying, allowing the system to retrieve and process data quickly. Proper indexing and relational structuring enable faster search operations, especially when dealing with large volumes of incident reports and user activities.

In addition, the database structure promotes easier maintenance, as the separation of data into well-defined tables makes it simpler to update, extend, or modify the system without affecting unrelated components.

The major entities identified in the database include Users, Roles, Incidents, Assignments, Comments, and Status History. Each entity plays a specific role within the system and contributes to the overall functionality of the application.

The Users entity stores information about individuals who interact with the system, including administrators and general users. The Roles entity defines different access levels and permissions, enabling role-based access control throughout the application.

The Incidents entity represents the core of the system and stores all reported incident data, including descriptions, locations, categories, timestamps, and current status. This entity is central to the system's functionality, as it captures the primary information being managed.

The Assignments entity is used to allocate incidents to specific administrators or response teams responsible for handling them. This ensures accountability and proper tracking of incident resolution processes.

The Comments entity allows users and administrators to communicate or provide additional information regarding specific incidents. This enhances collaboration and improves the quality of incident resolution.

The Status History entity tracks all changes made to the status of an incident over time. This provides a detailed audit trail, allowing the system to monitor the progression of each incident from reporting to resolution.

Relationships between these entities are established using primary keys and foreign keys, ensuring proper linkage and referential integrity across the database. These relationships define how data in one table is connected to data in another, enabling efficient data retrieval and maintaining the logical structure of the system.

Overall, the database design provides a strong foundation for the Community Incident Reporting System by ensuring structured data storage, maintaining integrity, and supporting efficient system performance.

4.2.1 ER Diagram Explanation

The Entity Relationship (ER) Diagram for the Community Incident Reporting System represents the logical structure of the database and illustrates how different entities interact with each other within the system. It provides a visual representation of the database design, showing entities, their attributes, and the relationships that exist between them. The ER diagram plays a crucial role in ensuring that the database is well-structured, normalized, and capable of supporting the functional requirements of the application.

At the core of the system is the Users entity, which stores information about all individuals who interact with the system. Each user is uniquely identified using a primary key (User_ID). The Users entity is linked to the Roles entity through a foreign key relationship, allowing each user to be assigned a specific role such as administrator or general user. This relationship supports role-based access control, ensuring that users can only access functionalities that correspond to their assigned roles.

The Roles entity defines the different categories of users within the system and helps enforce authorization rules. A one-to-many relationship exists between Roles and Users, meaning that one role can be assigned to multiple users, but each user can only have one role.

The central entity in the system is the Incidents entity. This entity stores all information related to reported incidents, including title, description, location, category, timestamp, and status. Each incident is uniquely identified using a primary key (Incident_ID). The Incidents entity is connected to the Users entity through a foreign key relationship, indicating that each incident is reported by

a specific user. This creates a one-to-many relationship where one user can report multiple incidents, but each incident is associated with only one reporting user.

The Incidents entity is also linked to the Assignments entity. This relationship is used to assign incidents to administrators or response teams responsible for handling them. Each assignment record contains details such as the assigned user, assignment date, and status of the assignment. This forms a one-to-many relationship where one incident can have one or multiple assignments over time, depending on the workflow of the system.

The Comments entity is related to the Incidents entity to facilitate communication and collaboration. Users and administrators can add comments to specific incidents to provide updates, clarifications, or additional information. This creates a one-to-many relationship between Incidents and Comments, as each incident can have multiple comments, but each comment belongs to a single incident.

Another important entity is the Status History entity, which tracks all changes made to the status of an incident over time. This entity records the previous status, new status, date of change, and the user responsible for the update. The relationship between Incidents and Status History is one-to-many, as a single incident can undergo multiple status changes throughout its lifecycle. This ensures traceability and provides an audit trail for monitoring incident progression from creation to resolution.

In addition, the database design includes a relationship between Users and Comments, as users are responsible for creating comments. Each comment is linked to the user who made it, forming a one-to-many relationship where one user can make multiple comments, but each comment is authored by a single user.

Similarly, the Assignments entity is linked to the Users entity to indicate which administrator or responder is assigned to handle a particular incident. This relationship ensures accountability and proper tracking of responsibilities within the system.

Overall, the ER diagram clearly illustrates how entities interact through well-defined relationships enforced using primary and foreign keys. These relationships ensure data integrity, eliminate redundancy, and support efficient data retrieval. The design also reflects a normalized structure that enhances scalability, maintainability, and consistency of the database, making it suitable for the operational needs of the Community Incident Reporting System.

4.2.3 Data Integrity

Data integrity within the Community Incident Reporting System was ensured through a combination of database design principles and application-level validation mechanisms. The goal of these measures is to maintain accurate, consistent, and reliable data throughout the lifecycle of the system.

Primary Keys

Each table in the database is designed with a unique primary key, typically named id, which serves as the main identifier for each record. The primary key ensures that every record within a table can be uniquely identified and accessed without ambiguity. This is fundamental for efficient querying and for establishing relationships between different tables in the database.

Foreign Keys

Foreign keys are used to enforce relationships between related tables and maintain referential consistency across the database. These keys ensure that relationships between entities are valid and that linked records exist in their respective parent tables.

Examples of foreign key usage in the system include:

- role_id in the Users table, which links each user to a specific role in the Roles table
- incident_id in the Incidents table, which uniquely identifies each incident record
- incident_id in the Assignments table, which associates assignments with specific incidents
- user_id in the Comments table, which links each comment to the user who created it

These relationships ensure that all dependent records remain logically connected and prevent inconsistencies in the dataset.

Constraints

To further enforce data accuracy, the database implements several constraints, including:

- NOT NULL constraints, which ensure that essential fields must always contain a value before a record can be saved
- UNIQUE constraints, which prevent duplicate values in fields that require distinct entries, such as email addresses or usernames
- CHECK constraints, which enforce specific rules on data values, such as acceptable ranges or predefined formats
- DEFAULT values, which automatically assign predefined values to fields when no input is provided

These constraints collectively ensure that only valid and well-structured data is stored in the database.

Referential Integrity

Referential integrity rules are applied to maintain consistency between related tables when records are updated or deleted. These rules prevent the existence of orphan records—records that reference non-existent entries in other tables. Depending on the relationship, actions such as CASCADE, RESTRICT, or SET NULL are used to control how changes in parent tables affect related child records.

Input Validation

In addition to database-level constraints, the system implements validation at the application level using the Laravel framework. This ensures that invalid or malicious data is rejected before it reaches the database.

Examples of validation rules include:

- Required fields validation, ensuring mandatory inputs are provided
- Email validation, ensuring correct email formatting
- Password rules, enforcing minimum security requirements such as length and complexity
- Image validation, ensuring uploaded files meet type and size restrictions
- GPS coordinate validation, ensuring latitude and longitude values fall within valid geographic ranges

Overall, these combined mechanisms ensure a high level of data integrity across the system. By enforcing rules at both the database and application levels, the system maintains accuracy, consistency, and reliability of all stored information, thereby supporting the effective operation of the Community Incident Reporting System.

4.2.4 Entity Relationship Diagram (ERD)

The Entity Relationship Diagram illustrates the relationship among the database tables.

The major relationships include:

- One User can report many Incidents.
- One Category can contain many Incidents.
- One Incident can have many Images.
- One Incident can have many Status History records.
- One User can receive many Notifications.

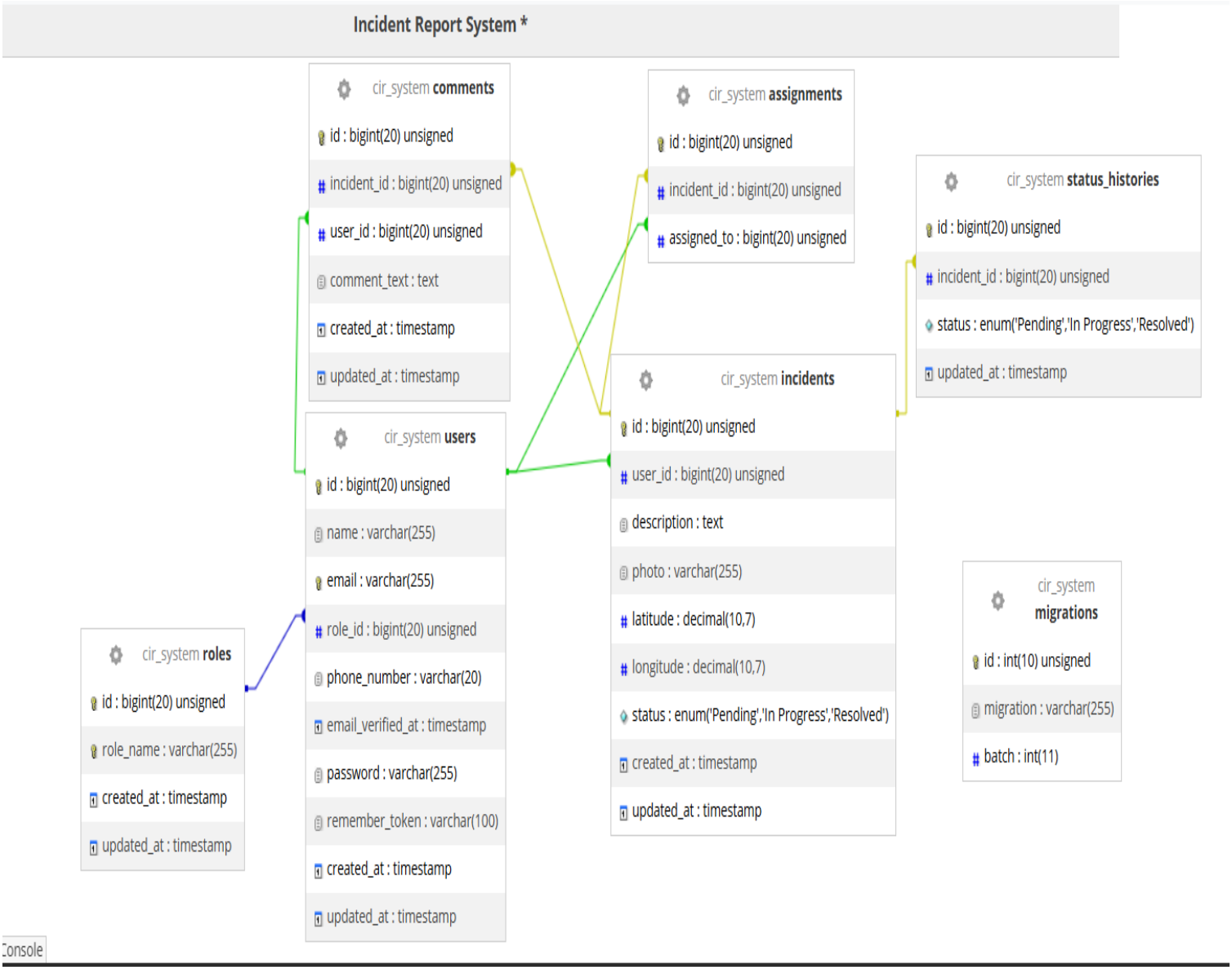


Figure 4.1 Entity Relationship Diagram

4.3 User Interface Design

The Community Incident Reporting System provides different interfaces based on user roles.

The interfaces were designed to be simple, responsive, and user-friendly.

4.3.1 Login Interface

The login page allows registered users to authenticate before accessing the system.

Features include:

- Email
- Password
- Login button
- Create Account link

Error messages are displayed whenever invalid credentials are entered.

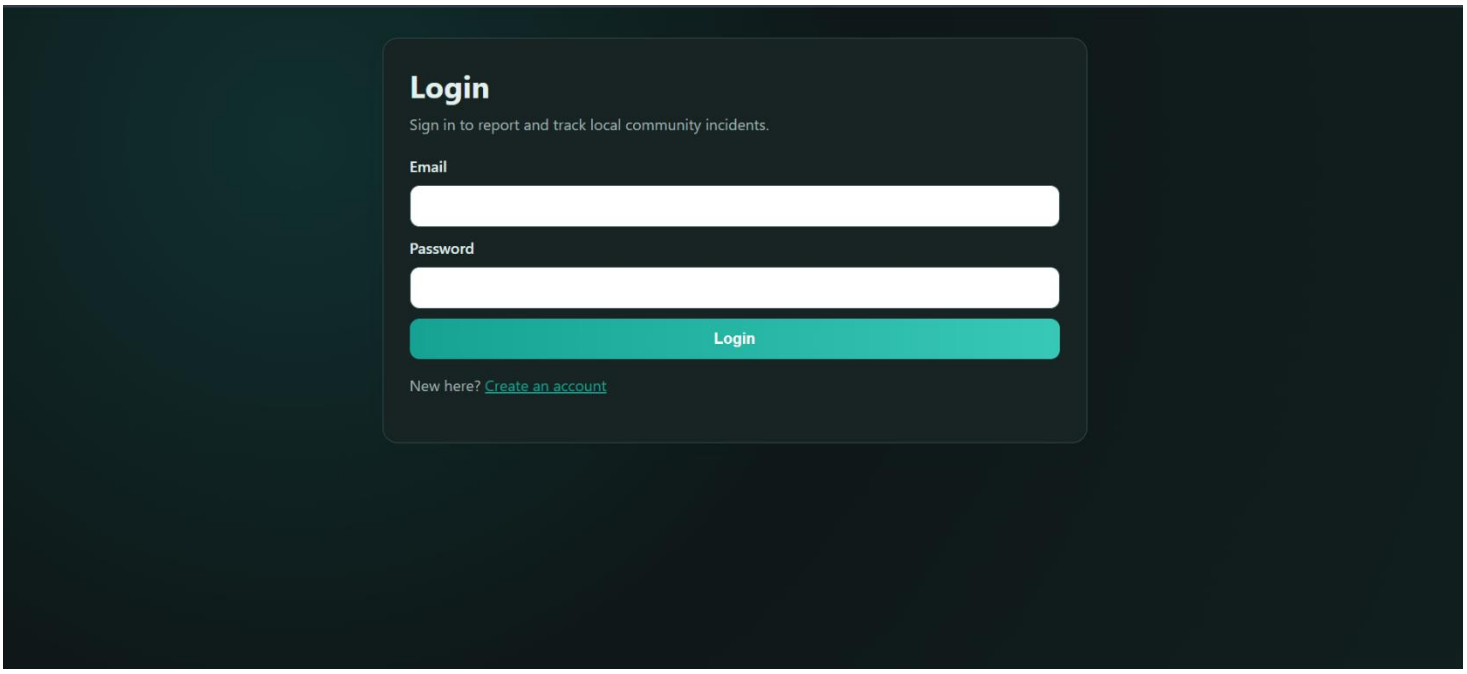


Figure 4.2 Login Page

4.3.2 Registration Interface

The registration page enables new users to create accounts.

Information collected includes:

- Name
- Email

- Phone Number
- Password
- Confirm Password

Validation messages guide users during registration.

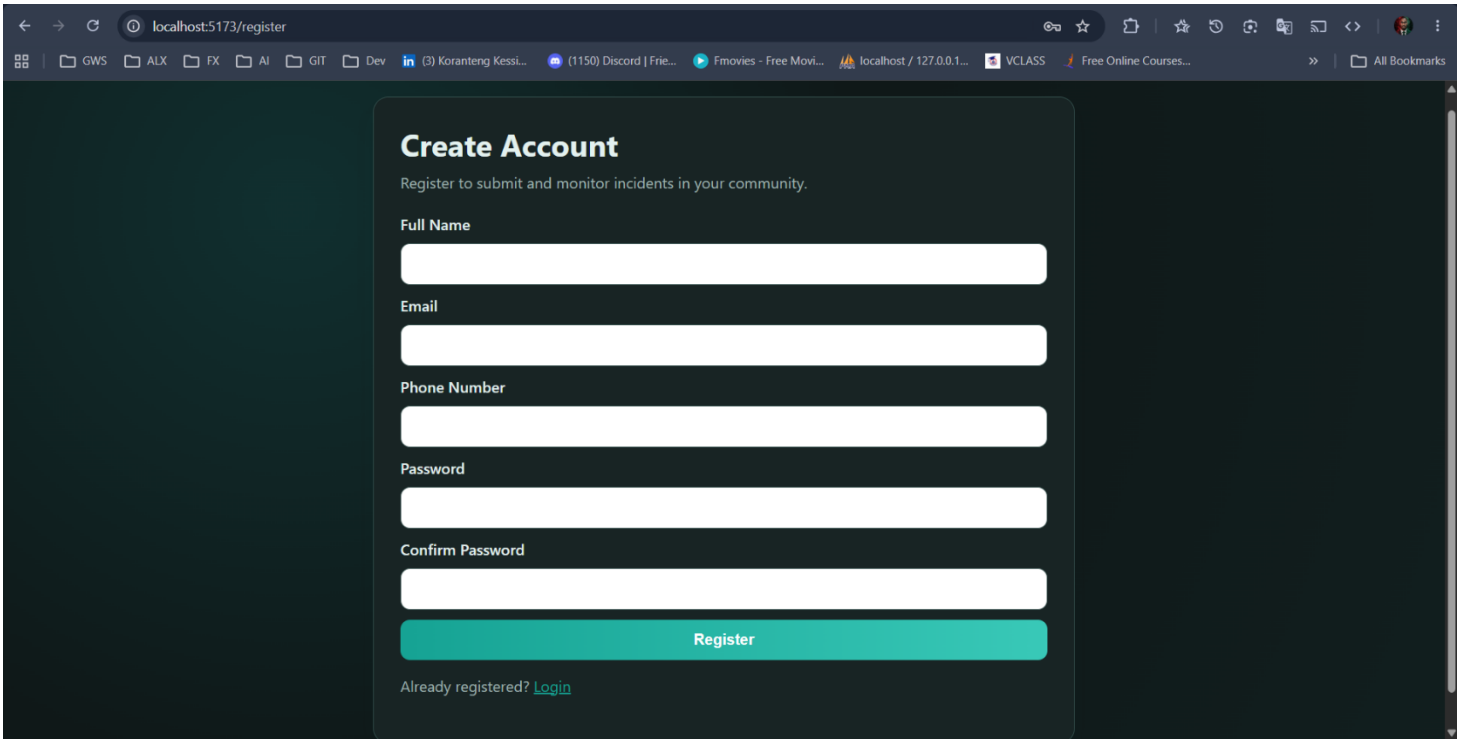


Figure 4.3 Register Page

4.3.3 User Dashboard

The user dashboard provides analytics for each user role. The dashboard also provides quick navigation through sidebar menus.

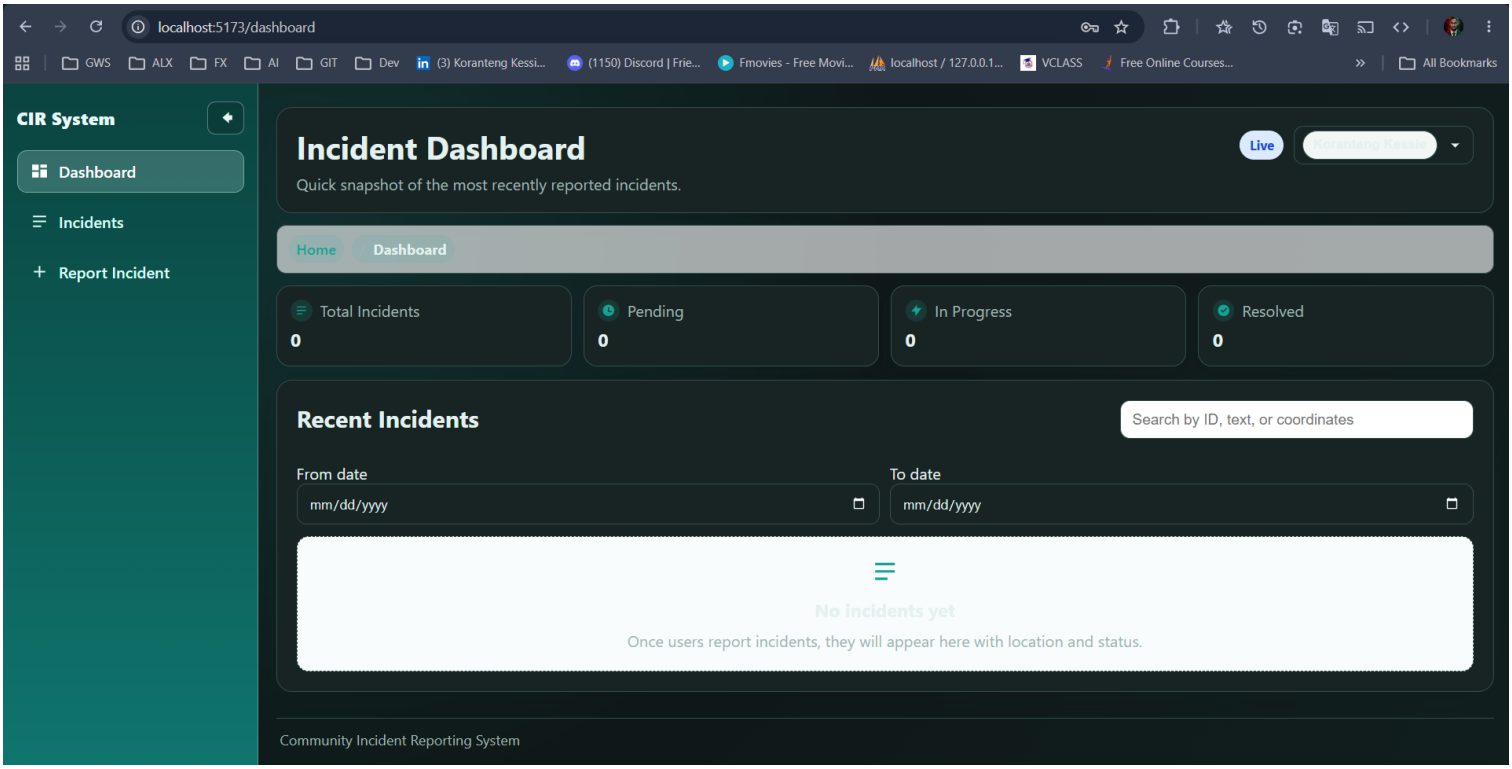


Figure 4.4 Citizen Dashboard

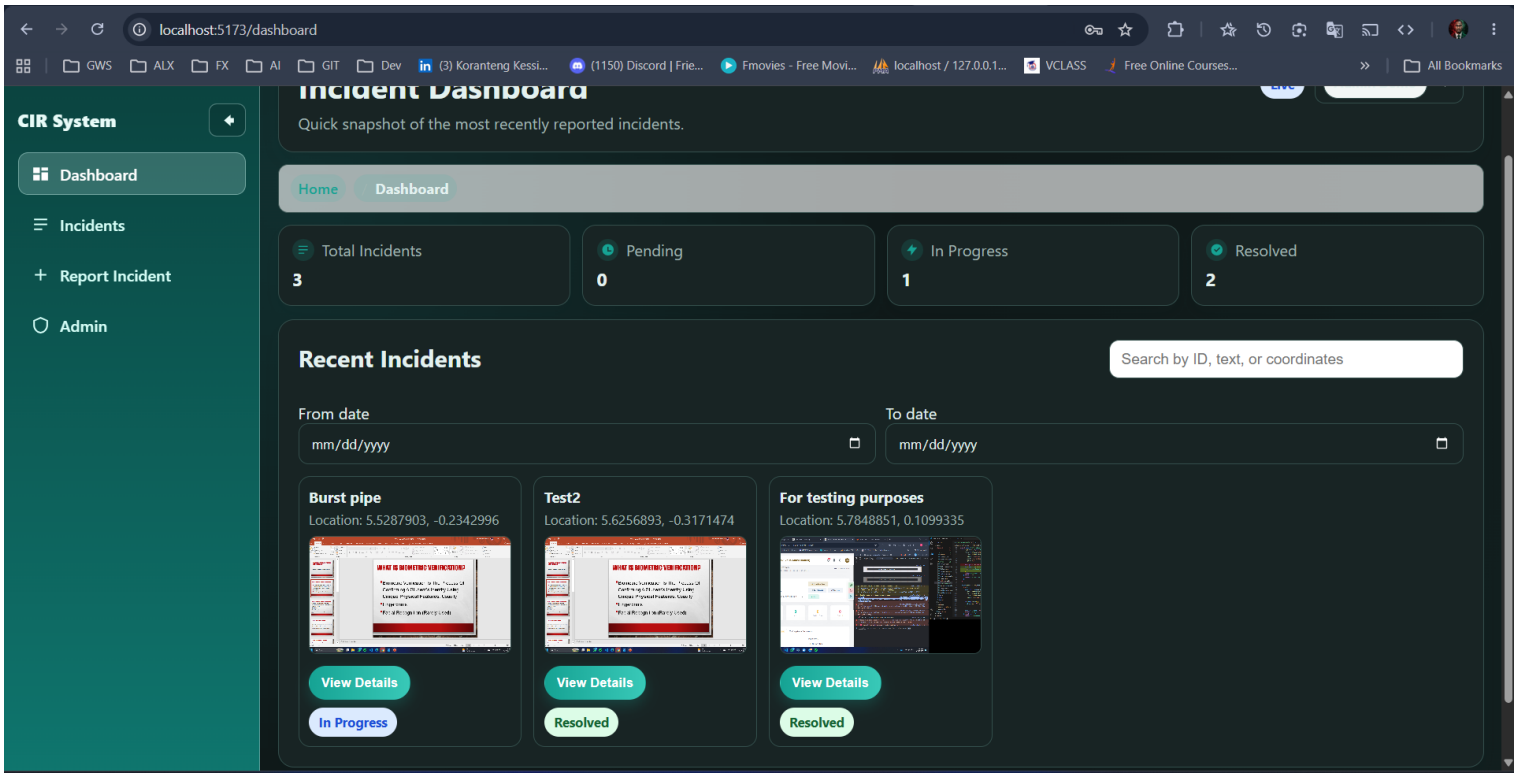


Figure 4.5 Admin Dashboard

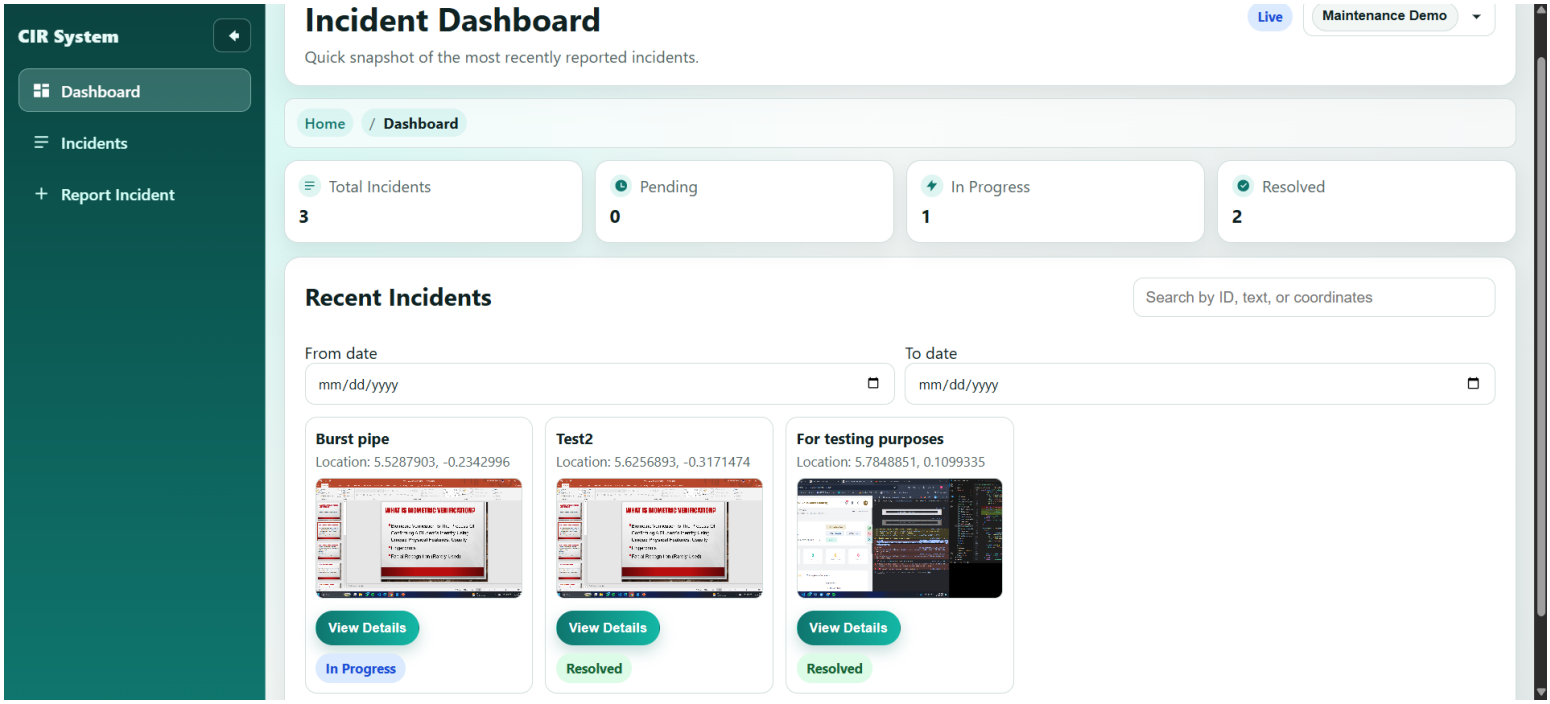


Figure 4.6 Maintenance Team Dashboard

4.3.4 Incident Reporting Interface

This interface contains:

- Description
- GPS coordinates
- Interactive map
- Image upload
- Submit button

The system validates all required fields before submission.

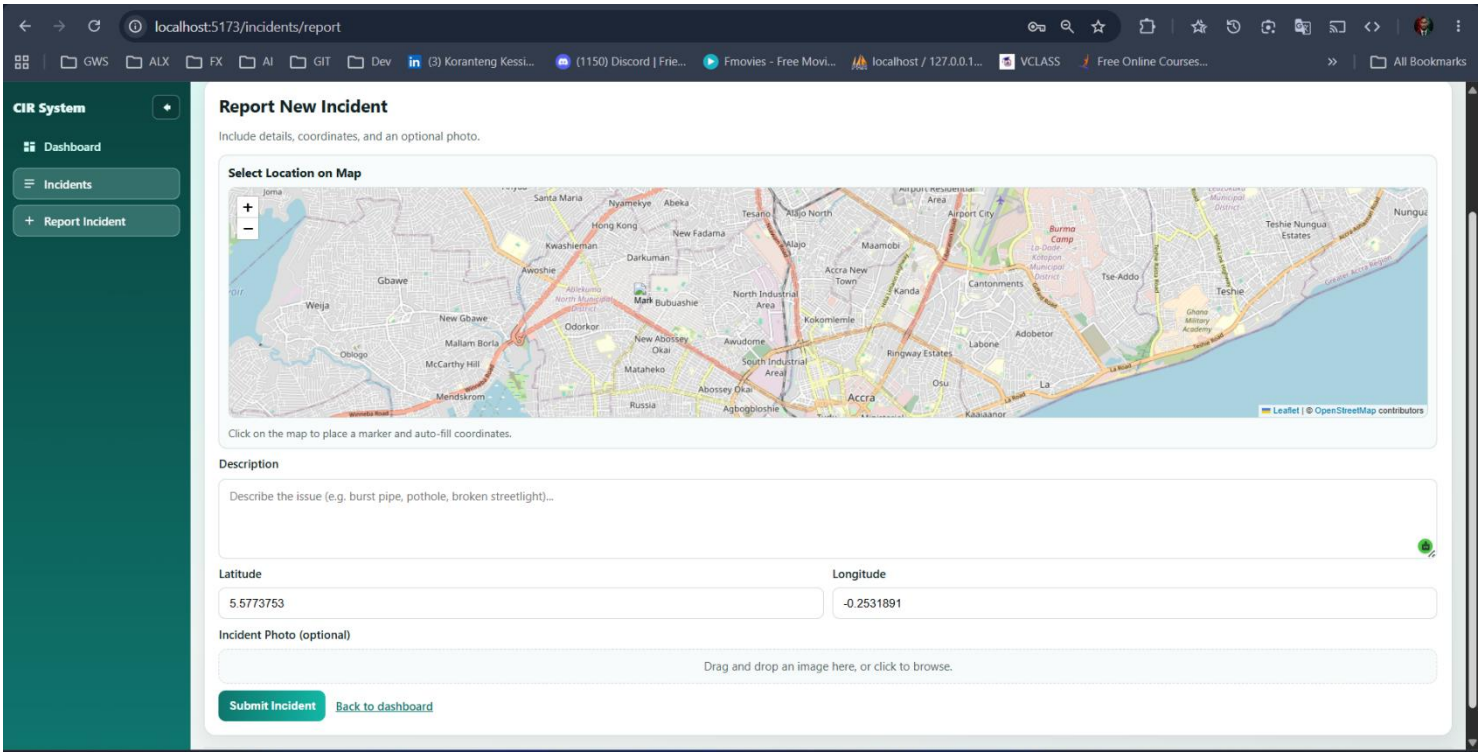


Figure 4.7 Incident Reporting Interface

4.3.5 Error Messages

The system provides meaningful feedback to users.

Examples include:

- Invalid login credentials
- Required field missing
- Invalid image format
- Network connection error
- GPS location unavailable

These messages assist users in correcting errors without ambiguity.

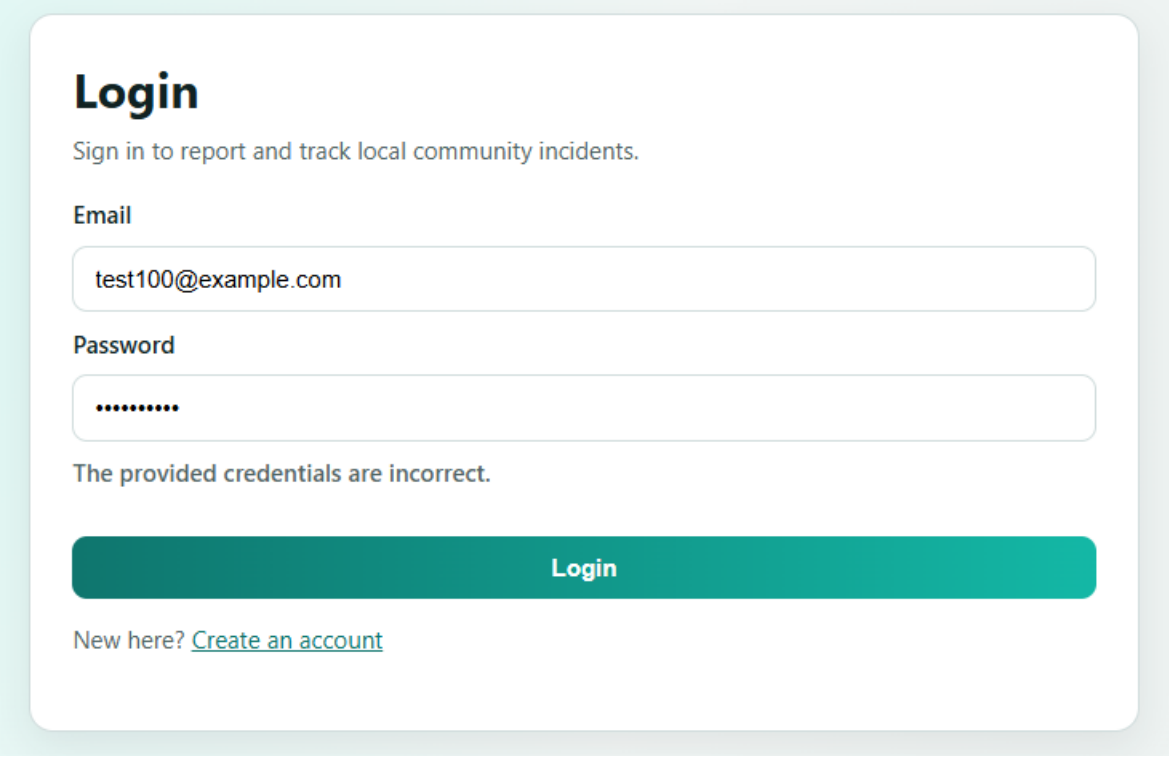


Figure 4.7 Invalid Login Credential Error Message

4.4 Summary

This chapter presented the system implementation environment and the technologies used in developing the Community Incident Reporting System. It provided a detailed explanation of the frontend, backend, database, and supporting tools that form the foundation of the application.

React.js was discussed as the frontend framework used to build a responsive and interactive user interface, while Laravel was selected as the backend framework due to its MVC architecture, security features, and strong support for RESTful API development. MySQL was identified as the relational database management system responsible for storing and managing all system data, including users, incidents, comments, assignments, and notifications.

In addition, the chapter highlighted the integration of external and development tools such as the Google Maps API for geospatial functionality, Visual Studio Code as the development environment, Git and GitHub for version control and collaboration, and Google Chrome for testing and debugging.

Furthermore, the database design was explained in detail, covering entity relationships, normalization principles, and how different tables interact through primary and foreign keys. Data integrity mechanisms were also discussed, including constraints, referential integrity rules, and input validation implemented using Laravel to ensure accuracy, consistency, and reliability of stored data.

Overall, the chapter demonstrated how the selected technologies and design principles work together to form a secure, scalable, and efficient system that meets the functional requirements of the Community Incident Reporting System.